

GROVE Pipeline — Process Documentation

grove_pipeline.py + LLM_evaluate.py

PART 1 — grove_pipeline.py

① Data Loading

Load games.csv into a DataFrame and read shared_task.jsonl line-by-line. Each JSON line (box score) is assigned to the matching game row by index.

In: games.csv, shared_task.jsonl

Out: games_df with game_data column



② Prompt Construction

Build the prompt using build_prompt(). Injects: TEXT_ID, the generated story, and the box score. Also appends annotation guidelines and PydanticOutputParser format instructions to force a strict JSON output.

In: text_id, story, game_data

Out: Filled prompt string



③ LLM Inference

Apply chat template, tokenise, and run llm.generate() with greedy decoding. Model runs 4-bit NF4 quantised across 2 GPUs. Only the newly generated tokens are decoded — the prompt is sliced off.

In: Prompt string

Out: Raw LLM output string



④ JSON Extraction

Use regex re.search(r'[\.*\|]', ...) to strip any conversational text the LLM added around the JSON array.

In: Raw LLM string

Out: Clean JSON array string



⑤ Pydantic Parse

Pass the JSON string to PydanticOutputParser. Validates field types (TEXT_ID str, SENTENCE_ID int, TOKENS List[str]). Wrong type → exception → falls back to empty list [].

In: JSON string

Out: List[Annotation] Python objects



⑥ Build Doc Token Map

Tokenise the story: split on sentence boundaries [!?.], then whitespace. Assign each word a global doc ID (1-indexed across the whole story), a sentence ID, and a local sentence token ID.

In: GENERATED_TEXT story

Out: doc_map e.g. {18: {sent_id:1, tok_id:18, token:'Wednesday'}}

⑦ Find Token Span

Scan doc_map sequentially to find where the LLM's TOKENS list appears verbatim. Returns the start and end global doc positions.

In: doc_map + TOKENS list

Out: DOC_TOKEN_START, DOC_TOKEN_END

⑧ Save Results CSV

Accumulate all annotations and save. Each row = one error annotation with its token positions, type, correction, model name, and time taken.

In: All annotations

Out: results_{model}.csv

PART 2 — LLM_evaluate.py

⑨ Load token_lookup.yaml

Load the master index. doc_to_sent maps a global doc token ID → sentence ID + local token ID. sent_to_doc does the reverse. Used to align GSML (has both schemes) with LLM results (DOC only).

In: token_lookup.yaml

Out: token_lookup dict

⑩ create_mistake_dict() × 2

Parse both GSML and LLM results into nested dicts keyed by text_id → doc_start_idx. Each mistake's token range is stored as a Python set (e.g. {54,55,56}). For LLM results, missing SENT indices are resolved via doc_to_sent. Overlapping spans raise an Exception immediately.

In: gsml.csv + results_{model}.csv + token_lookup

Out: gsml_dict, submitted_dict

⑪ check_token_ids()

Open each raw .txt story file. For every annotation, assert the token string at the stored DOC position actually matches the stored TOKENS field. Guards against tokenisation drift between pipeline and evaluation.

In: texts/ directory + mistake dicts

Out: Confirmed alignment (or assertion error)

⑫ Mistake-level Matching

For each GSML mistake, check if any submitted mistake's token set intersects it. If yes → True (recalled). Consume-on-match: a submitted annotation can only fire once, preventing one large span from claiming multiple gold mistakes.

In: gsml_dict, submitted_dict

Out: `per_category_matches` {NAME: {S001_18: True/False}, ...}



⑬ Token-level Matching

Build a flag map of every token in every story {gsml:F, submitted:F}. Flip flags to True for each annotation span, running once for GSML and once for LLM results. Gives partial credit where mistake-level gives zero.

In: `gsml_dict`, `submitted_dict`, `token_lookup`

Out: `document_tokens` with per-token match flags



⑭ Metrics + eval_{model}.csv

Compute 4 metrics overall and per category (NAME, NUMBER, WORD, CONTEXT, NOT_CHECKABLE, OTHER): Recall = correct recalls / GSML total | Precision = correct recalls / submitted total
Token Recall = matched tokens / GSML tokens | Token Precision = matched tokens / submitted tokens

In: `per_category_matches`, `document_tokens`

Out: `eval_{model}.csv`